Evaluation Metrics for LLMs, RAG & Agentic Systems

A compact reference of concepts, metrics and practices for building and testing a RAG pipeline

1. Why evaluation is different for LLMs

LLM outputs are open-ended and non-deterministic, so there is rarely a single correct string to compare against. Evaluation should happen at three levels: **component** (retriever, generator, individual tool), **end-to-end** (final answer quality and task success) and **production** (latency, cost, user feedback). Keep **offline** evaluation (a fixed golden dataset run in CI before every release) separate from **online** evaluation (live traffic, A/B tests, drift monitoring). Metrics are either **reference-based**, comparing the output with a gold answer, or **reference-free**, judging quality from the query, retrieved context and output alone.

2. Core LLM (generation) metrics

Exact Match and Accuracy check whether the output equals the gold answer; they suit closed tasks such as classification or short-answer QA but are brittle for free text. **Precision, Recall and F1** measure token or label overlap (P = TP/(TP+FP), R = TP/(TP+FN), F1 = 2PR/(P+R)) and are standard for extractive QA and entity extraction. **BLEU** (precision-oriented, used for translation), **ROUGE** (recall-oriented, used for summarization) and **METEOR** score n-gram overlap with a reference, but they penalize valid paraphrases. **BERTScore** and embedding cosine similarity capture semantic closeness and are far more tolerant of rewording. **Perplexity**, the exponent of the average negative log-likelihood, shows how well a model predicts text; it is useful for language-model training but says little about task quality. **Pass@k** is the probability that at least one of k generated code samples passes the unit tests, as in HumanEval.

LLM-as-a-Judge uses a strong model with a rubric to give pairwise preferences or 1-5 scores for helpfulness, tone and coherence. It scales well but must be calibrated against human labels because of position, verbosity and self-preference bias. **Human evaluation** remains the gold standard, using Likert ratings, pairwise preference (win-rate or Elo) and inter-annotator agreement (Cohen's or Fleiss' kappa).

Beyond overlap scores, rate each output on **quality dimensions**: correctness, faithfulness (groundedness), relevance, completeness, coherence, fluency, conciseness, tone, instruction following and format validity such as a JSON schema. Add **safety** checks (toxicity, bias, PII leakage, jailbreak and prompt-injection resistance, refusal and over-refusal rate) and **robustness** checks (consistency across paraphrases, typos and temperature settings). Public benchmarks such as MMLU (knowledge), GSM8K and MATH (reasoning), HumanEval and SWE-bench (code), TruthfulQA (truthfulness) and HELM (broad suites) track general ability, but they do not replace a custom domain evaluation set, and data contamination can inflate their scores.

3. RAG evaluation: retrieval plus generation

A RAG failure is either a **retrieval** failure (wrong or missing context) or a **generation** failure (the model ignores or distorts the context). Score the two stages independently so you know exactly what to fix.

A. Retrieval metrics (need labelled relevant chunks per query)

Recall@k is the fraction of relevant documents found in the top k results, and **Hit rate** asks whether at least one was found; this is the most important retrieval metric, because anything not retrieved cannot be used by the LLM. **Precision@k** is the fraction of the top k that is relevant and reflects how much noise reaches the model. **MRR** (mean reciprocal rank) averages 1/rank of the first relevant result and fits cases where one good chunk is enough. **nDCG@k** uses graded relevance with a rank discount, rewarding systems that place the best chunks first, while **MAP** averages precision across all relevant results per query. LLM-judged **context precision** (are relevant chunks ranked high?) and **context recall** (are all needed facts present?) give reference-light variants, as in the Ragas framework.

B. Generation metrics in RAG (the RAG triad)

Faithfulness (groundedness) is the share of claims in the answer that the retrieved context supports; extract the claims, verify each against the context, and you detect hallucination. Citation accuracy and attribution belong here too. **Answer relevancy** asks whether the answer actually addresses the question, often estimated by generating questions from the answer and comparing their embeddings with the original query. **Context relevance** checks that the retrieved material is pertinent to the query, and **answer correctness** compares facts with the gold answer. **Noise robustness and abstention** test whether the system says it does not know when the context lacks the answer. Useful tools include Ragas, DeepEval, TruLens, LangSmith, Phoenix and promptfoo. Build a golden set of 50-200 real queries plus synthetic ones, include unanswerable and adversarial cases, tune chunk size, overlap, embedding model, top-k and reranker, and re-run the suite after every change.

4. Agentic system evaluation

An agent plans, calls tools, observes results and iterates, so failures can occur at any step. Evaluate both the **final outcome** and the **trajectory**, meaning the sequence of thoughts, tool calls and observations.

Task success is measured by completion rate and goal accuracy; **pass^k** (all k runs succeed) captures reliability rather than luck. **Tool use** is scored by tool selection accuracy, argument correctness, schema validity and the rate of redundant or hallucinated calls, usually against an expected call or by checking the execution result. **Planning and reasoning** look at plan quality, step correctness, trajectory precision, recall and in-order match against a reference path, and goal drift. **Efficiency** covers steps and tool calls per task, tokens, latency, cost per successful task, and loop or retry counts. **Robustness** measures error recovery, handling of tool failures and timeouts, and resistance to ambiguous inputs or prompt injection hidden in tool outputs. **Memory and context** checks retention of earlier facts and state consistency across turns. **Safety and control** tracks policy violations, unauthorized or destructive actions, data exfiltration and the human-in-the-loop escalation rate. In **multi-agent** systems also measure handoff success, coordination overhead and role adherence, and in **conversational** agents track turn-level relevance, resolution rate, containment and user satisfaction.

Common agent benchmarks are SWE-bench (software issues), WebArena and Mind2Web (web navigation), GAIA (general assistants), AgentBench, tau-bench (tool-agent-user with policies), BFCL (function calling) and OSWorld (computer use). Typical methods are programmatic state verification (did the database or file actually change?), trajectory comparison, LLM judges over traces, simulated users, replay of production traces and sandboxed environments for repeatability.

5. Operational and business metrics

Track **latency** (time to first token, tokens per second, end-to-end p50/p95/p99) and **cost** (tokens in and out, cost per query and per successful task, cache hit rate). Monitor **reliability** (error, timeout and tool-failure rates, rate-limit hits, uptime) and **quality drift** after model, prompt or data updates, including drift in embeddings and query distribution. Collect **user signals** such as thumbs up or down, edit distance of accepted outputs, regeneration rate, escalation to a human and CSAT, and trace every span (retrieval, LLM, tool) with OpenTelemetry-style observability.

6. LLM-as-a-judge: best practices and pitfalls

Give the judge a clear rubric with defined score anchors, ask it to reason before scoring, and use structured output. Prefer pairwise comparison or binary pass/fail checks over fine 1-10 scales. Mitigate position bias by swapping answer order, reduce verbosity bias and self-preference by using a judge from a different model family, and validate the judge on 50-100 human-labelled examples, reporting agreement. Use judges for subjective or reference-free criteria, and use deterministic checks (regex, JSON schema, unit tests, exact match) wherever possible because they are cheaper and more reliable.

7. Practical evaluation workflow for your RAG system

First, create a versioned golden set with question, gold answer and gold source chunks, covering easy, multi-hop, unanswerable and adversarial queries, split into dev and test. **Second**, evaluate retrieval with Recall@k, MRR and nDCG, and tune chunking, embeddings, hybrid search (BM25 plus vector), reranking and query rewriting. **Third**, evaluate generation with faithfulness, answer relevancy, correctness, citation accuracy and abstention, and tune the prompt, context order and model. **Fourth**, if the system is agentic, add tool-call accuracy, trajectory and task-success metrics plus cost and step budgets. **Fifth**, run everything in CI as regression gates with thresholds against a baseline. **Sixth**, monitor production by sampling traces, running judges online, collecting user feedback and feeding failures back into the golden set.

Report confidence intervals (bootstrap), use enough samples (a few hundred), run multiple seeds, and apply significance tests such as a paired t-test or McNemar before claiming one system beats another. Optimizing a single metric can harm another, so track a balanced scorecard of **quality, safety, latency and cost**.